

scoring questions in both rule-defined and open-ended areas. For the instruction I , the generated response to be evaluated o , and the scoring problem q , if q is verifiable by rules, we first use the LLM to automatically extract segments e of o , which is related to scoring question q . Subsequently, we use the rule R_q written for q to obtain the evaluation result $r_q \in \{0, 1\}$, that is:

$$e = \mathcal{M}_{ext}(I, q, o) \quad (1)$$

$$r_q = R_q(e) \quad (2)$$

where $\mathcal{M}_{ext}$ indicates the LLM with the prompt used for extraction. Otherwise, if q cannot be verified by rules, we directly use the LLM to measure the quality of o :

$$r_q = \mathcal{M}_{eva}(I, q, o) \quad (3)$$

where $\mathcal{M}_{eva}$ denotes the LLM with the prompt used for evaluation. For composition types, considering that their satisfaction is a prerequisite for satisfying some constraints, we model the dependencies of its scoring questions. Specifically, for *Chain*, all the scoring questions of the subsequent task depend on the answers to those of the preceding task. And for *Selection*, all the scoring questions of the selection branch depend on whether the correct selection branch is selected. If a scoring question is judged as "no", all the scoring questions depending on it will also be directly judged as "no". Formally, we denote the set of scoring questions that q depends on as $Dep(q)$. After all scoring questions have been independently verified, *Dependency Aggregation* will be performed, and the result of q will be calculated as follows:

$$r'_q = r_q \bigwedge_{p \in Dep(q)} r_p \quad (4)$$

Finally, following InfoBench [7], we calculate Decomposed Requirements Following Ratio (DRFR) as the final score during *Score Aggregation*. Considering a benchmark dataset has N instructions, the instruction i has m_i scoring questions, and the result of the j -th scoring question is r'_{ij} , the metric is calculated as: $DRFR = \sum_{i,j} r'_{ij} / \sum_i m_i$. Figure 5 shows a framework of our evaluation protocol.

4.3 Benchmark Statistics

COMPLEXBENCH contains 1,150 instructions and 5,306 scoring questions, as shown in Table 2. Nesting depth means the maximum depth of composition types. In addition to three basic composition types including *And*, *Chain*, and *Selection*, we adopt a separate category whose instructions simultaneously contain *Chain* and *Selection*, aiming to use these two challenging types to explore the boundary of LLMs' ability in complex instruction-following³. We also present the task distribution of COMPLEXBENCH in Appendix C.

Category	Nesting Depth	#Inst.	#Len.	#Ques.	#Con.
And	1	475	279.39	4.09	4.14
Chain	1	70	352.11	4.83	4.94
	2	170	486.84	6.24	6.32
Selection	1	80	753.15	2.91	2.06
	2	224	664.13	4.40	3.09
	≥ 3	46	1409.93	5.76	3.78
Selection & Chain	2	30	440.37	4.37	3.63
	≥ 3	55	398.82	6.18	5.27
Overall	-	1150	477.51	4.61	4.19

Table 2: Statistics of COMPLEXBENCH including the number of instructions (**#Inst.**), the average number of characters (**#Len.**), scoring questions (**#Ques.**), and constraints (**#Con.**) per instruction.

5 Experiments

5.1 Agreement Evaluation

To measure the agreement between our evaluation method and manual evaluation, we randomly sample 200 instructions from COMPLEXBENCH to construct a meta-evaluation dataset. Five LLMs are involved in this evaluation as generation models. We employ GPT-4-1106 [39] as our primary judge and adopt two metrics to confirm the reliability of our method: (1) **Overall Pairwise Agreement**: Given an instruction, two model responses (denoted as A and B), the human annotators are instructed to compare the quality and choose from 3 options, namely A better than B, tie, B better than A. Subsequently, the automatic evaluation scores for two model responses are converted into pairwise

³Since *And* commonly appears in various instructions, we simply categorize instructions containing both *Chain* / *Selection* and *And* together with those only containing *Chain* / *Selection* into one category.

comparisons to measure agreement with human annotators. (2) **Question-level Agreement:** Given an instruction and a model response, human annotators are instructed to judge whether each scoring question is satisfied respectively. Then, we calculate the agreement between automatic evaluation results and human-annotated ones.

For the Overall Pairwise Agreement, we sample 500 pairs from the outputs of 5 LLMs. Direct Scoring serves as a baseline, which adopts a scoring prompt [5] to assign a score to the response with a scale of 1-10. As shown in Table 3, our method can improve the agreement with manual evaluations compared to Direct Scoring with a large margin. *Dependency Aggregation* also shows its important contribution to our method due to its modeling of composition structures.

Evaluation Method	Pairwise Agreement
Ours	0.614
Ours w/o <i>Dep.</i>	0.574
Direct Scoring	0.512

Table 3: Overall Pairwise Agreement with human. *Dep.* means *Dependency Aggregation*.

For the Question-level Agreement, the scoring questions in the meta-evaluation dataset are categorized into two types: (1) Rule-defined, which can be verified by rules and constitutes 17% of the total, and (2) Open-ended, which is not verifiable by rules. We compare our method with Direct Scoring, which considers a response with a score above 5 to satisfy all scoring questions of an instruction. We also remove rule arguments (w/o rule) to verify its effectiveness. As shown in Table 4, RAL outperforms all the baselines and exhibits an impressive 87.82% agreement with humans at the overall level. The LLM-based evaluator (i.e., RAL w/o rule in Table 4) shows its weakness in rule-defined areas that rule arguments mainly contribute to, supporting our motivation.

Subset	Evaluator	Agreement between human
Rule-defined	RAL	95.36%
	RAL w/o rule	82.02%
	Direct Scoring	62.02%
Open-ended	RAL	86.28%
	RAL w/o rule	86.28%
	Direct Scoring	77.83%
Overall	RAL	87.82%
	RAL w/o rule	85.56%
	Direct Scoring	75.18%

Table 4: Question-level Agreement with human.

5.2 Automatic Evaluation

5.2.1 Setup

We use GPT-4-1106 [39] as our judge to evaluate 15 LLMs: (1) **Closed-source LLMs:** GPT-4-1106, Claude-3-Opus [40], GLM-4 [41], ERNIEBot-4, GPT-3.5-Turbo-1106. (2) **Open-source LLMs:** Qwen1.5-Chat [42], Llama3-Instruct [43], InternLM2-Chat [44], Baichuan2-Chat [45], Mistral-Instruct [46], InternLM2-Chat [44], ChatGLM3-Chat [47]. The sizes of these models vary from 6B to 72B. We use greedy search for reproducibility, and the maximum generation length is 8,192.

5.2.2 Main Results

The main results are shown in Table 5. **Firstly**, the widely recognized powerful GPT-4 still fails to complete 20% of complex instructions, highlighting the necessity of complex instruction evaluation. **Secondly**, as the complexity of composition types within instruction increases, the performance of all LLMs significantly drops, especially on *Selection* and *Chain*. This aligns with our motivation for constructing complex composition types. **Thirdly**, the performance of most open-source LLMs falls short compared to closed-source LLMs especially on complex composition types, indicating that open-source LLMs still have a large room for improvement in chasing the capabilities of closed-source LLMs.

To dissect the ability of LLMs to follow specific constraint and composition types, we calculate the average accuracy of scoring questions for

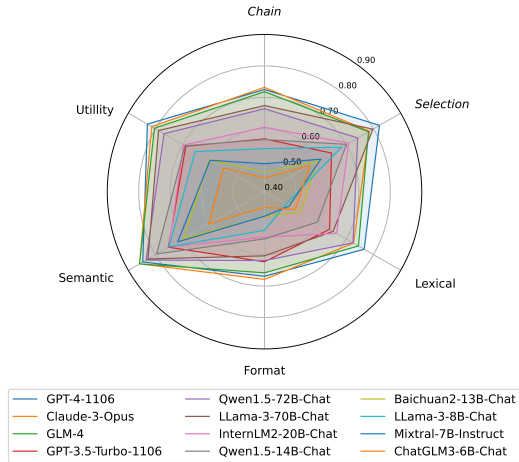


Figure 6: The performance of LLMs on different constraint and composition types.

Category	And		Chain		Selection				Selection & Chain			All
Nesting Depth	1	1	2	Avg.	1	2	≥ 3	Avg.	2	≥ 3	Avg.	Avg.
<i>Closed-Source Language Models</i>												
GPT-4-1106	0.881	0.787	0.759	0.766	0.815	0.772	0.694	0.765	0.802	0.626	0.675	0.800
Claude-3-Opus	0.886	0.784	0.779	0.780	0.764	0.749	0.592	0.724	0.695	0.576	0.609	0.788
GLM-4	0.868	0.763	0.739	0.745	0.768	0.739	0.626	0.724	0.809	0.647	0.692	0.779
ERNIEBot-4	0.866	0.749	0.735	0.738	0.725	0.696	0.649	0.692	0.756	0.600	0.643	0.764
GPT-3.5-Turbo-1106	0.845	0.686	0.630	0.644	0.661	0.561	0.475	0.561	0.565	0.482	0.505	0.682
<i>Open-Source Language Models</i>												
Qwen1.5-72B-Chat	0.873	0.749	0.730	<u>0.735</u>	<u>0.751</u>	0.698	0.521	0.675	0.611	0.521	0.546	0.752
Llama-3-70B-Instruct	0.858	<u>0.769</u>	0.722	0.733	0.747	<u>0.704</u>	<u>0.675</u>	<u>0.706</u>	0.573	<u>0.571</u>	<u>0.571</u>	<u>0.757</u>
InternLM2-20B-Chat	0.796	0.666	0.648	0.652	0.648	0.599	0.543	0.597	0.611	0.488	0.522	0.678
Qwen1.5-14B-Chat	0.817	0.657	0.636	0.641	0.622	0.621	0.536	0.606	0.550	0.435	0.467	0.680
Baichuan2-13B-Chat	0.760	0.583	0.517	0.533	0.571	0.479	0.404	0.480	0.443	0.409	0.418	0.591
Llama-3-8B-Instruct	0.778	0.669	0.568	0.592	0.597	0.552	0.483	0.546	0.626	0.429	0.484	0.638
Mistral-7B-Instruct	0.737	0.574	0.556	0.560	0.554	0.493	0.411	0.488	0.534	0.374	0.418	0.592
Qwen1.5-7B-Chat	0.802	0.598	0.611	0.608	0.519	0.564	0.570	0.558	<u>0.634</u>	0.491	0.531	0.658
InternLM2-7B-Chat	0.755	0.633	0.598	0.607	0.532	0.568	0.525	0.555	0.550	0.432	0.465	0.634
ChatGLM3-6B-Chat	0.701	0.556	0.490	0.506	0.455	0.430	0.411	0.431	0.573	0.312	0.384	0.546

Table 5: DRFR of LLMs computed by our proposed RAL method. The highest performance among open-source models is underlined, while the highest performance overall is **bold**.

each type. The results are shown in Figure 6. **Firstly**, for constraints, LLMs generally perform better on Semantic and Utility constraints but struggle with the Format and Lexical constraints that have explicit evaluation standards. **Secondly**, for compositions, *Chain* presents severe challenges while *Selection* come second. We speculate that the main difficulty in *Selection* lies not only in choosing the correct branch but in executing it without interference from irrelevant branches. More results and analyses are in Appendix I.

5.2.3 Analysis

Decomposition of instructions with composition types. To explore whether decomposing complex instructions and executing them through multi-round interactions can improve the performance of LLMs, we manually decompose COMPLEXBENCH instructions based on composition types (e.g., *Chain* into sequential tasks, *Selection* into selection and execution branches, while *And* remains intact) and compare the performance of LLMs between executing decomposed instructions step-by-step and original instructions in one step. The scoring questions of original instructions are split into corresponding decomposed ones with the same dependencies to ensure a fair comparison.

Table 6 shows that GPT-3.5-Turbo-1106 generally performs worse in decomposed instructions, especially as the complexity of composition types within instructions increases. We conjecture that this is due to cumulative errors in multi-round interactions, highlighting that our benchmark is challenging and cannot be simply solved via instruction decomposition.

The Coherent Test for Selection. To comprehensively measure the performance of LLMs on different conditions of *Selection*, we merge the instructions with the same branches and selection functions but different conditions into the same task group. For example, the instruction about the Mona Lisa shown in Table 1 and another instruction where everything else remains the same except the final condition "Painting: Mona Lisa" is changed to "Painting: Galloping horse" are merged into the same task group. The two instructions need to execute two different selection branches. We calculate the proportion of instructions with all scoring questions correct (*Original Test*) and group tasks with all scoring questions correct (*Coherent Test*). Formally, considering that there are N instructions containing *Selection*, they are divided into K task groups. Each instruction i has m_i scoring questions, and the result

Category	Nesting Depth	Origin	Decomposition	Δ
And	1	0.845	0.845	0.000
	2	0.686	0.655	-0.031
Chain	1	0.686	0.655	-0.031
	2	0.630	0.583	-0.047
	≥ 3	0.661	0.631	-0.030
Selection	1	0.661	0.631	-0.030
	2	0.561	0.520	-0.041
	≥ 3	0.475	0.411	-0.064
Selection & Chain	2	0.565	0.504	-0.061
	≥ 3	0.482	0.415	-0.067
Overall	-	0.682	0.652	-0.030

Table 6: The performance of GPT-3.5-Turbo-1106 on original and decomposed instructions.